

Exercise Sheet 10

Lab Sessions: July 9/10, 2026

In this exercise sheet, we look at reproducibility engineering when working with Large Language Models (LLMs). Calling an LLM raises three practical questions: how do we hand the container an API key *without* risking to leak it, how do we make a model's output as *reproducible* as possible, and how do we make sure the output has the *structure* we expect? We work with a small model (Gemma 3 1B) served locally by [llama.cpp](#), so everything runs on the CPU requiring only a few GB of RAM.

1 Preparation

For this exercise, we provide files in the [RepEng FIMGit Repository](#) (directory `LabSession10/`). Update your local repository that you cloned in the first lab session:

1. Open a terminal in your local repository.
2. Pull the latest changes using `git pull`.

All tasks use Docker and Docker Compose, which you already have installed from the previous lab sessions.

2 Keeping a Secret

When you call a remote LLM, you authenticate with an API key. Because the key is tied to billing and access, it must be kept secret: it should *never* be hard-coded in the source code, copied into an image, or committed to git. Instead, we inject it into the container *at run time*. In `LabSession10/secrets/` we provide three small demos that each hand a container the same (dummy) key three different ways. They run offline, so no real key or network is needed.

2.1 Secrets in an Environment Variable

The first approach passes the key as an environment variable. Inside the directory `LabSession 10`, enter `secrets/1-env-var/` and store the API key as a variable:

Terminal Session

bash

```
user@host$ cd secrets/1-env-var
user@host$ export OPENAI_API_KEY=sk-demo-1234567890abcdef
```

- (a) Build the image from the Dockerfile and add the tag `env-demo-img`. Then run the container, name it `env-demo-1`, and pass the stored variable as an environment variable by adding `-e` to the run command.
- (b) The container reads the key from its environment. Now inspect where that value is visible. What does the following command reveal?

Terminal Session

bash

```
user@host$ docker inspect env-demo-1 2>/dev/null | grep -i openai
```

- (c) Run the container using Docker Compose.
- (d) Inspect again where the key value is visible. What does the following command reveal?

Terminal Session

bash

```
user@host$ docker inspect env-demo 2>/dev/null | grep -i openai
```

2.2 Secrets in a .env File

Putting the key on the command line (or into `compose.yaml`) is easy to leak – it lands in your shell history or in a file you might commit. An `.env` file keeps the value in one place. Enter `secrets/2-dotenv/` and store the API key in a `.env` file by copying it from the provided file `.env.example`.

Terminal Session

bash

```
user@host$ cd secrets/2-dotenv
user@host$ cp .env.example .env
```

- (a) Run the container using Docker Compose.
- (b) Inspect where the API key is visible. What does the following command reveal?

Terminal Session

bash

```
user@host$ docker inspect dotenv-demo 2>/dev/null | grep -i openai
```

- (c) Compare this with the previous approach: inside the container, is the key stored any differently than in Exercise 2.1? What did the `.env` file improve?

2.3 Secrets in a Mounted Secret File

In this approach, we mount the key as a read-only *file* and have the application read that file. Enter `secrets/3-mounted-file/`, create a directory `secrets/` and store an API key `sk-mounted-file-demo-1234567890abcdef` in a file `secrets/openai_api_key`.

- (a) Run the container using Docker Compose.
- (b) Inspect where the API key is visible. What does the following command reveal?

Terminal Session

bash

```
user@host$ docker inspect mounted-file-demo-1 2>/dev/null | grep -i \
openai
```

- (c) Inside the container, is the key stored any differently than in the previous exercises? What did the `openai_api_key` file improve?

2.4 Preventing Accidents

The approaches in Exercises 2.2 and 2.3 do not help if the secret slips into git or into the image by accident.

- (a) Which files in `secrets/` must never be committed, and how do we enforce that?
- (b) How do we keep the same files out of the Docker build context (and thus out of the image)?

3 Reproducible LLM Outputs

By default, an LLM's output is *non-deterministic*: the same prompt can produce different output each time. In this exercise, we look at two request parameters that help us make the output deterministic: `temperature` and `seed`.

3.1 Starting the Model Server

The directory `LabSession10/llm/` contains a `docker-compose.yml` that downloads a small LLM (Gemma 3 1B, quantised to Q4_K_M) and serves it with `llama.cpp`. Its API is addressable via the OpenAI specification, so we can use the `openai` Python client. Specifically, we are using the [Chat Completions API](#) in this exercise.

If you are working on narrow-sea, the server is already running.

If you are working locally, set up the container as follows:

Terminal Session

bash

```
user@host$ cd LabSession10/llm
user@host$ docker compose up -d
```

In both cases, the server listens on port 11434 (i.e., the address is `127.0.0.1:11434`).

Next, set up the Python client code:

Terminal Session

bash

```
user@host$ python -m venv .venv && . .venv/bin/activate
user@host$ pip install -r requirements.txt
```

In the following, we use the command `python`. On narrow-sea, use `python3`. On your own machine, it depends on its configurations.

3.2 Temperature and Seed

Experiment with different settings for the two parameters affecting the determinism of the output that we discussed in class: *temperature* and *seed*

The script `llm/generate.py` sends the same prompt to the model several times and prints each response. It accepts `--temperature/-t`, `--seed/-s`, and `--iterations/-i`.

3.2.1 Experiments

Run the following experiments and discuss these questions:

- What does the temperature do?
- What does the seed do?
- How do temperature and seed interact?

(a) Set the temperature to 0 and vary the seed:

Terminal Session

bash

```
user@host$ python generate.py -t 0 -i 3
user@host$ python generate.py -t 0 -s 1 -i 3
user@host$ python generate.py -t 0 -s 999 -i 3
```

(b) Do *not* set a temperature, but fix a seed:

Terminal Session

bash

```
user@host$ python generate.py -s 42 -i 3
```

(c) Fix a seed and change the temperature:

Terminal Session

bash

```
user@host$ python generate.py -s 42 -t 0.7 -i 3
user@host$ python generate.py -s 42 -t 1.5 -i 3
```

4 Structured Outputs

Often we need the model's answer in a specific shape, e.g., a JSON instance matching a given schema. There are two ways to achieve this. We can put the schema *in the prompt* and hope the model complies, or we can use *constrained decoding* (*structured outputs*), where the tool restricts generation to tokens allowed by a grammar derived from the schema, so the output is valid by construction. Both variants are provided in the directory `LabSession10/structured_outputs/`; the server from Exercise 3 must be running.

Both scripts ask for the smallest number that satisfies the same schema:

```
schema
{
  "type": "integer",
  "anyOf": [
    { "type": "integer", "minimum": 3, "maximum": 1 },
    { "type": "integer", "minimum": 12000, "maximum": 12002 }
  ]
}
```

The first branch (`minimum: 3, maximum: 1`) can never be satisfied, so the only valid instances are 12000, 12001, and 12002; the smallest valid answer is 12000.

4.1 Schema in the Prompt

Run the prompt-only variant a few times:

```
Terminal Session bash
user@host$ python schema_in_prompt.py -i 5
```

Does the model always return a valid instance? Why is that not guaranteed?

4.2 Constrained Decoding

Now run the structured-outputs variant, which passes the schema via `response_format` so the server enforces it during decoding:

```
Terminal Session bash
user@host$ python structured_output.py -i 5
```

How does the result differ, and what exactly is now guaranteed?

4.3 oneOf vs. anyOf

Constrained decoding is only as reliable as the translation from schema to grammar, and not every tool supports every JSON Schema feature. We provide two schemas in `structured_outputs/schemas/` that are identical except that one uses `oneOf` and the other `anyOf`. Convert both to llama.cpp's GBNF grammar with the provided [converter script](#) and compare:

Terminal Session

bash

```
user@host$ python json_schema_to_grammar.py schemas/number_oneof.json \  
              > oneof.gbnf  
user@host$ python json_schema_to_grammar.py schemas/number_anyof.json \  
              > anyof.gbnf  
user@host$ diff oneof.gbnf anyof.gbnf
```

What does the diff tell you, and what does it imply for reproducibility?

5 Reproducibility and LLMs (Antwort-Wahl-Verfahren)

In this task, each question has exactly one correct answer. Mark the correct answer.

- (a) You send the following request to the *local* llama.cpp server (using a CPU for inference) several times on the same machine:

request.py

```
client.chat.completions.create(  
    model="gemma3:1b",  
    messages=[{"role": "user", "content": "Name a colour."}],  
    temperature=0.0,  
)
```

Ignoring hardware/driver changes, are the responses reproducible (identical) across runs?

- ☐ No – an LLM’s output is always random.
 - ☐ Yes – at `temperature=0` the result is deterministic on a local system if the model runs on a CPU.
 - ☐ Only if a `seed` is also set.
 - ☐ Only on a GPU.
 - ☐ None of these options
- (b) A model call uses constrained decoding with a JSON Schema that includes numeric `minimum/maximum` bounds and a `oneOf`, and it returns successfully. Which statement is correct?
- ☐ The instance is guaranteed to satisfy the full schema, regardless of the tool used.
 - ☐ The instance is only guaranteed to match the schema features the tool actually supports.
 - ☐ The call cannot succeed, because `minimum` is never supported.
 - ☐ Structured outputs guarantee that the value is semantically correct.
 - ☐ None of these options